Operating Systems: Memory Management & Virtual Memory Architecture

Paging, Multi-Level Page Tables, TLB Translation, Page Faults, and Replacement Algorithms

Doc Id: OS-MEM-2024-002 | **Practice:** Memory Management & Virtual Memory | **System:** x86-64 MMU, Linux Virtual Memory Subsystem, Page Cache | **Year:** 2024

1. Virtual Address Translation & Multi-Level Page Tables

Virtual memory provides each process with the illusion of a contiguous, private address space, decoupling logical addresses from fragmented physical DRAM frames.

On x86-64 architectures, standard 48-bit virtual addressing is mapped to physical memory via 4-level paging (PML4 -> PDPT -> PD -> PT -> 4KB Physical Page Frame).

• **Paging Overhead:** Each translation traversal through all 4 page table levels requires 4 sequential memory reads. To mitigate this latency, hardware incorporates the **Translation Lookaside Buffer (TLB)**, a fast associative cache.

• **Huge Pages:** To reduce TLB miss pressure on memory-intensive workloads, modern OS support 2MB (Level 2) and 1GB (Level 1) Huge Pages.

Paging Structure	Page Size	Virtual Bits Covered	Table Entries	TLB Coverage / Entry
Standard Page (4-Level)	4 KB	Bits [11:0] offset	512 entries per level	4 KB per TLB slot
Transparent Huge Page (THP)	2 MB	Bits [20:0] offset	Bypasses Page Table L1	2 MB (512x standard coverage)
Gigabyte Huge Page	1 GB	Bits [29:0] offset	Bypasses PD and PT	1 GB (262,144x standard coverage)
5-Level Paging (PAGING57)	4 KB	57-bit address space	Adds PML5 level	Enables up to 128 PB virtual memory

2. Page Fault Handling and Demand Paging Workflow

Under demand paging, pages are only loaded into physical DRAM when referenced by the processor:

1. The CPU attempts memory access at virtual address; MMU checks TLB. On TLB miss, page table walk occurs.

2. The valid/present bit in the Page Table Entry (PTE) is 0 -> MMU triggers hardware Interrupt 14 (**Page Fault Exception**).

3. CPU switches to kernel mode ring 0, pushes faulting address to CR2 register, and executes page fault handler do_page_fault().

4. Kernel verifies memory reference against Virtual Memory Areas (VMA). If invalid -> sends SIGSEGV.

5. If valid reference, kernel allocates free physical frame. If physical memory is full, a page replacement algorithm chooses a victim frame and writes back to swap if dirty.

6. Kernel schedules non-volatile disk I/O to read the required page into the frame.

7. Once I/O completes, PTE is updated with physical frame address and valid bit set to 1. The instruction is re-executed transparently.

Access Latency Comparison: L1 Cache Hit: ~1 ns (4 cycles) | TLB Hit: ~1.5 ns | Main Memory DRAM Access: ~60-80 ns | SSD Page Fault Swap: ~50,000-100,000 ns (50-100 μs) | HDD Page Fault Swap: ~8,000,000 ns (8 ms). A single page fault incurs a 100,000x latency penalty!

3. Page Replacement Algorithms: Fault Rates and Belady's Anomaly

When physical memory is exhausted, the OS replacement policy selects which frame to evict.

Belady's Anomaly: The phenomenon where increasing the number of physical page frames results in an INCREASE in the number of page faults. FIFO is susceptible to Belady's Anomaly, whereas stack algorithms (LRU, Optimal) are mathematically immune.

Algorithm	Hardware Overhead	Page Fault Rate	Susceptible to Belady's?	Implementation Approach
Optimal (MIN / Clairvoyant)	Impossible (requires future knowledge)	Minimum theoretical	No (Stack algorithm)	Benchmark baseline for offline trace analysis
Least Recently Used (LRU)	High (timestamp / hardware doubly linked list)	Near-optimal	No (Stack algorithm)	Maintains exact access recency order
Clock / Second-Chance	Low (1 reference bit per PTE)	Very close to LRU	No	Circular buffer pointer; clears ref bit on sweep
First-In, First-Out (FIFO)	Minimal (FIFO queue of arrival)	Poor / Unstable	Yes (demonstrated with 3 vs 4 frames)	Evicts oldest loaded page regardless of usage
Least Frequently Used (LFU)	High (frequency counter per page)	Poor under phase changes	No	Evicts lowest reference count; suffers from history
Two-Queue (2Q / Linux Page Cache)	Low (Active + Inactive LRU lists)	Superior scan resistance	No	Filters one-time sequential scans into inactive list

4. Thrashing and The Working Set Model

If the sum of working sets across all active processes exceeds available physical DRAM (Σ WSS_i > Total RAM), processes spend more time paging in and out than executing actual instructions. This condition is termed **Thrashing**.

• The CPU utilization plummets toward zero as all cores become blocked waiting for swap disk I/O.

• The OS resolves thrashing using the **Working Set Model (Peter Denning)**: monitor process reference string over parameter window Δ. If process cannot fit its Working Set Size into assigned frames, the OS suspends the process entirely and swaps its entire state out to restore system stability.